

UNIVERSIDAD DEL VALLE DE GUATEMALA

Ing. Pablo Andrés Barreno Koch



Analizador Semántico

Bryan Alberto Martínez Orellana 23542

Adriana Sophia Contreras Palacios 23044

Brandon Werner Rivera Cabrera 23088

GUATEMALA

31 de agosto de 2026

Repositorio

https://github.com/Bamo0507/Compilador_Kotlin

Descripción General

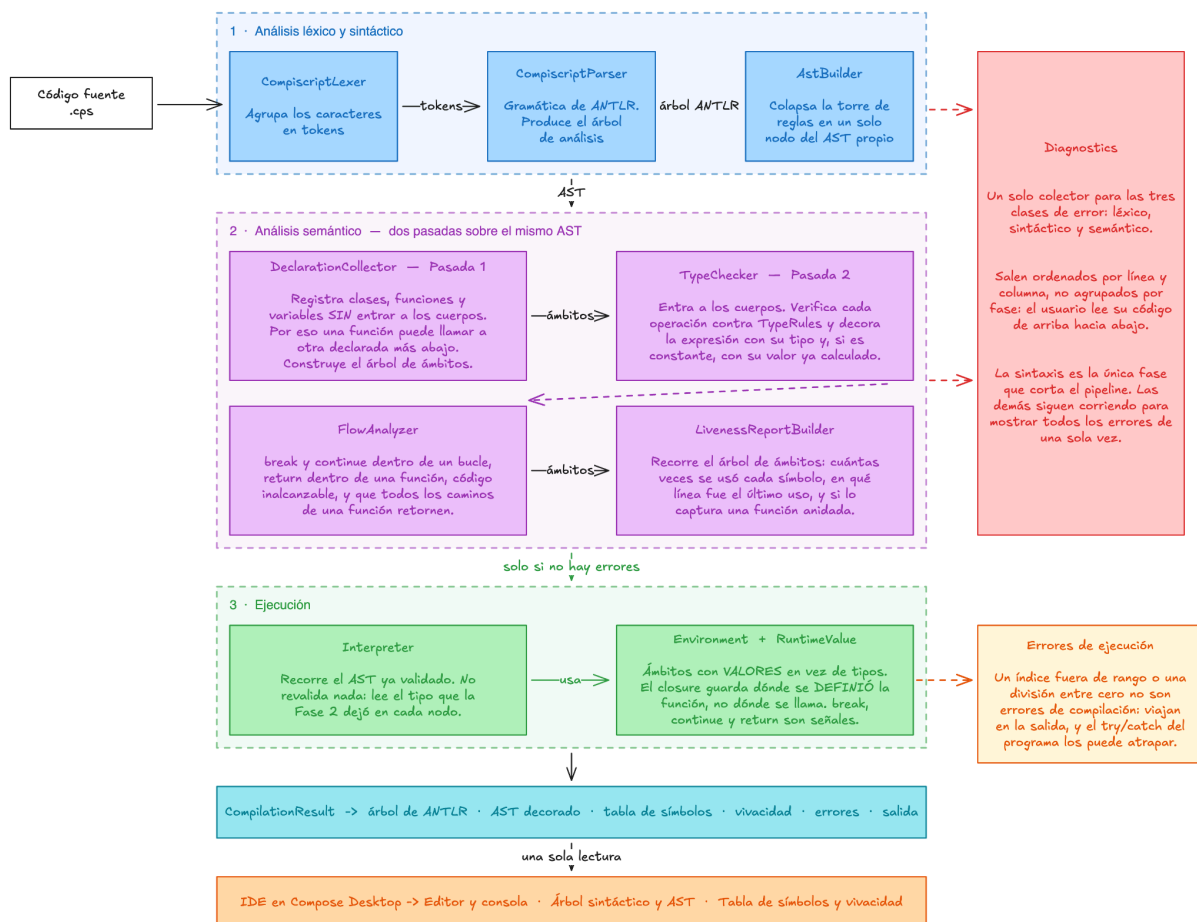
El proyecto consiste en un analizador semántico para Compiscript, un lenguaje basado en un subconjunto de TypeScript, construido sobre Kotlin y usando ANTLR para el análisis léxico y sintáctico. Adicional, se desarrolló un IDE de escritorio con Compose Multiplatform que permite escribir un programa, compilarlo y ver todo lo que el compilador produce: el árbol sintáctico, el árbol propio ya validado, la tabla de símbolos y el resultado de ejecutar.

La idea que guió toda la construcción es que el compilador no es una sola pasada que lo hace todo, sino una cadena de fases donde cada una agrega información sobre la misma estructura en vez de construir una nueva. Es decir, el árbol de sintaxis abstracta se construye una única vez y de ahí en adelante las fases lo van decorando: primero se le anotan los ámbitos y los símbolos, luego los tipos, después los valores constantes que se pueden resolver desde compilación, y al final el intérprete lo recorre confiando en todo eso.

Arquitectura de la Implementación

Compiscript — cómo funciona el compilador

Una llamada a `CompilerPipeline.compile(fuente)` encadena las tres fases.
Cada una agrega información al mismo AST en vez de construir uno nuevo.



El punto de entrada de todo es una sola función, `CompilerPipeline.compile(fuente)`, que encadena las tres fases y devuelve un único objeto `CompilationResult` con el árbol de ANTLR, el AST decorado, la tabla de símbolos, el reporte de vivacidad, los errores y la salida de ejecución. Se decidió hacerlo así para que la interfaz gráfica no tuviera que orquestar nada, es decir, la GUI llama una vez y de ahí en adelante solo lee campos, lo cual hace que agregar una pantalla nueva no implique tocar el compilador.

Análisis Léxico y Sintáctico

Para esta parte se reutilizó la gramática oficial de Compiscript en ANTLR, a la cual únicamente se le agregó el tipo float, ya que el enunciado lo exige en las operaciones aritméticas y la gramática entregada no lo contemplaba en baseType. A partir de esa gramática, ANTLR genera el lexer y el parser, y sobre ellos se construyeron dos piezas propias.

Primero, un SyntaxAnalyzer que corre el lexer y el parser sobre el texto y les conecta un listener de errores propio, de tal forma que los errores léxicos y sintácticos no se escriban a consola, que es lo que ANTLR hace por defecto, sino que se recojan en la misma estructura donde después caerán los errores semánticos.

Luego, un AstBuilder que es un visitor sobre el árbol de ANTLR y produce el árbol de sintaxis abstracta propio del proyecto. Se decidió no trabajar directamente sobre el árbol de ANTLR porque este arrastra toda la torre de reglas de precedencia de la gramática, es decir, para llegar a un simple identificador hay que bajar por expression, assignmentExpr, conditionalExpr, logicalOrExpr y varias más. Por ejemplo, para el programa por defecto del IDE el árbol de ANTLR tiene 553 nodos mientras que el AST propio tiene 76.

Análisis Semántico

Esta es la parte central del proyecto y se resolvió con dos pasadas sobre el mismo AST, decisión que considero la más importante de toda la implementación.

En la primera pasada, el DeclarationCollector recorre el programa registrando clases, funciones y variables, pero sin entrar a los cuerpos. Se hizo así para poder soportar las referencias hacia adelante, es decir, que una función pueda llamar a otra que está declarada más abajo en el archivo, o que una variable se declare con un tipo de clase que aparece después. Si se validara todo en una sola pasada eso sería imposible, ya que al llegar a la llamada el nombre todavía no existiría. En esta pasada también se construye el árbol de ámbitos y se detectan las redeclaraciones y los ciclos de herencia.

En la segunda pasada, el TypeChecker sí entra a los cuerpos y es donde se verifica el sistema de tipos completo: aritmética sobre integer y float, operadores lógicos sobre boolean, compatibilidad en comparaciones y asignaciones, cantidad y tipo

de argumentos en las llamadas, tipo de retorno, acceso a miembros de clase, uso de `this` y validación de listas e índices. Las reglas puras del sistema de tipos se separaron en un archivo aparte, `TypeRules`, que no conoce ni el AST ni los ámbitos y únicamente responde preguntas del tipo "¿cabe un valor de tipo X en algo declarado Y?", lo cual permitió probarlas sin construir un programa completo.

Un detalle que vale la pena mencionar es el manejo de las cascadas de errores. Cuando una expresión no se puede tipar, en vez de detenerse se le asigna un tipo especial `ErrorType`, y cualquier operación que involucre `ErrorType` no genera un error nuevo. Gracias a eso el pipeline puede seguir corriendo después del primer error y mostrarle al usuario todos sus problemas de una sola vez, en lugar de que los tenga que ir arreglando de uno en uno. Por ejemplo, un programa con una variable redeclarada y tres asignaciones de tipo incompatible reporta los cuatro errores en la misma corrida y no produce mensajes falsos adicionales.

Asimismo, el `TypeChecker` aprovecha el recorrido para hacer plegado de constantes, es decir, cuando una expresión se puede resolver desde compilación su valor queda guardado en el nodo. De tal forma que `print(3 + 5)` no ejecuta ninguna suma en tiempo de ejecución porque el 8 ya está en el árbol, y adicionalmente esto le sirve al análisis de flujo para reconocer que un `while (true)` o un `while (1 == 1)` son bucles infinitos.

Posteriormente corren dos análisis más sobre ese mismo árbol ya tipado. El `FlowAnalyzer` valida las reglas que dependen de dónde está una sentencia y no de qué tipos tiene: que `break` y `continue` estén dentro de un bucle, que `return` esté dentro de una función, que no haya código inalcanzable después de un salto, y que todos los caminos de una función con tipo de retorno efectivamente retornen. El `LivenessReportBuilder`, por su parte, recorre el árbol de ámbitos y arma un reporte de vivacidad que indica cuántas veces se usó cada símbolo, en qué línea fue su último uso y si lo captura una función anidada; esto responde a la petición de tener información sobre cuándo un símbolo ya no se va a volver a utilizar.

Tabla de Símbolos

La tabla de símbolos se modeló como un árbol de ámbitos y no como una pila. La diferencia es que en una pila los ámbitos se descartan al cerrarse, mientras que aquí cada ámbito queda colgando de su padre incluso después de que la fase

terminó de recorrerlo. Se hizo de esta forma porque el enunciado pide mostrar el estado de la tabla por cada entorno (global, función, clase y bloque), y si los ámbitos se descartaran al salir de ellos al final solo quedaría el global y no habría nada que mostrar.

Cada símbolo guarda su nombre, la categoría de declaración, el tipo resuelto, el offset dentro de su ámbito, la línea donde se declaró, si es miembro de una clase, si ya fue inicializado, y los contadores de uso que alimentan el reporte de vivacidad. Cabe mencionar que la categoría que se muestra en la interfaz se calcula combinando dos ejes del símbolo, ya que "campo" y "método" no son categorías distintas de "variable" y "función" sino la misma cosa viviendo dentro de una clase, y se prefirió no duplicar ese estado para que no se pudiera contradecir con el ámbito donde el símbolo realmente vive.

IDE

La interfaz se construyó con Compose Multiplatform para escritorio y tiene tres vistas. La primera es el editor con la consola de salida y la lista de errores, donde estos aparecen etiquetados por nivel (léxico, sintáctico o semántico) y ordenados por línea y columna en vez de agrupados por fase, ya que el usuario lee su código de arriba hacia abajo; al hacer clic sobre un error el editor resalta esa línea y hace scroll hasta ella.

La segunda vista dibuja los dos árboles, el de ANTLR y el AST propio, con la posibilidad de verlos lado a lado. Los dos se dibujan con el mismo componente porque ambos se convierten previamente a una vista neutral, de tal forma que el dibujante no sabe de dónde vino lo que está pintando. En el AST cada nodo de expresión muestra el tipo que le asignó la fase semántica y, si es constante, su valor ya calculado, lo cual convierte esa pantalla en la evidencia visual del trabajo del analizador.

La tercera vista es la tabla de símbolos, con el árbol de ámbitos navegable a la izquierda y los símbolos del ámbito seleccionado a la derecha, además de una pestaña con el reporte de vivacidad.

¿Cómo ejecutar?

El proyecto requiere Java 21 y usa Gradle con el wrapper incluido, por lo que no hace falta instalar nada más.

Para levantar el IDE:

```
./gradlew run
```